

AI-Powered Stock Price Scraper & Analytics Dashboard

Prepared By:

Radhil (Roll No: 2410998567)

Parth Doomra (Roll No: 2410998557)

CSE - AIFT, Chitkara University

ABSTRACT

This document describes the AI-Powered Stock Price Scraper & Analytics Dashboard project — requirements, modules, design placeholders, implementation code (backend), testing, and future enhancements. It is prepared for academic submission and contains test cases for both users and administrators.

FEATURES

- Automated real-time stock data collection using scraping and APIs.
- Interactive analytics and exportable reports.
- AI-assisted interpretation of market behaviour.
- Secure user authentication and session management.

MODULES

- User Authentication& Profile Management
- Stock Scraper Engine (scheduler + scraper)
- Data Processor and Storage (MySQL)
- Analytics & Reporting Engine
- Admin Panel & Logs

CODE IMPLEMENTATION (BACKEND)

```
from flask import Flask, jsonify
import requests
from bs4 import BeautifulSoup

app = Flask(__name__)

def get_stock_price(symbol):
    url = f'https://finance.yahoo.com/quote/{symbol}'
    resp = requests.get(url, timeout=10)
    soup = BeautifulSoup(resp.text, 'html.parser')
    price = soup.find('fin-streamer', {'data-field': 'regularMarketPrice'})
    return price.text if price else None

@app.route('/api/price/<symbol>')
def price(symbol):
    val = get_stock_price(symbol)
    return jsonify({'symbol': symbol, 'price': val})
```

TESTING

This section contains detailed test cases for User and Admin functional testing. Each test case includes: ID, Objective, Steps, Test Data, Expected Result, and Status (to be filled after execution).

Test Cases for Users

Test ID	Objective	Steps	Input Data	Expected Output	Status
TCU_01	User Registration	Register new user with valid credentials	Name, Email, Password	Registration successful; redirected to dashboard	Pass/Fail
TCU_02	Duplicate Registration	Attempt registration with existing email	Existing email	Error: Email already registered	Pass/Fail
TCU_03	Login	Login with valid username & password	Valid credentials	Dashboard loads successfully	Pass/Fail
TCU_04	Invalid Login	Login with incorrect password	Invalid credentials	Displays 'Invalid Username/Password'	Pass/Fail
TCU_05	Stock Search	Search for a listed stock symbol	Stock Symbol (e.g. AAPL)	Displays latest stock price	Pass/Fail
TCU_06	Empty Search	Search without entering any symbol	Empty field	Displays error: 'Enter valid stock symbol'	Pass/Fail
TCU_07	Fund Transfer	Transfer fund using valid Transaction ID	Amount, Transaction ID	Funds transferred successfully	Pass/Fail
TCU_08	Insufficient Balance	Try transferring more than available balance	Excessive amount	Displays 'Insufficient Balance'	Pass/Fail
TCU_09	Profile Update	Update user info (email/phone)	Valid details	Profile updated confirmation	Pass/Fail
TCU_10	Logout	Click logout from dashboard	-	Session terminated, redirect to login page	Pass/Fail

Test Cases for Admin

Test ID	Objective	Steps	Input Data	Expected Output	Status
TCA_01	Admin Login	Login with valid admin credentials	Admin ID, Password	Admin dashboard loads successfully	Pass/Fail
TCA_02	Add Event	Add a new event in system calendar	Event title, date	Event successfully added	Pass/Fail
TCA_03	View User Queries	Access user queries in support panel	-	Displays list of user messages	Pass/Fail
TCA_04	Send Email	Compose and send email to registered user	Recipient, Subject, Message	Email sent successfully	Pass/Fail
TCA_05	Manage Users	Deactivate an inactive account	User ID	User status updated to 'Inactive'	Pass/Fail
TCA_06	Fund Request Approval	Approve or reject user fund requests	Request ID	Displays confirmation of approval/rejection	Pass/Fail
TCA_07	System Monitoring	View logs from admin dashboard	-	Displays log data with timestamps	Pass/Fail
TCA_08	Backup Database	Trigger a manual database backup	-	Backup process starts and file created	Pass/Fail
TCA_09	Update Incentive Rule	Change incentive percentages for referrals	New values	Rules updated successfully	Pass/Fail
TCA_10	Generate Report	Export transaction data for month	Month/Year	Report file downloaded in CSV/PDF	Pass/Fail

Figure 1.1: Home page of the StockSense web application welcoming users to the Smart Stock Analysis Platform, featuring navigation links, sign-in options, and quick access buttons to begin exploring live stock data and analytics.

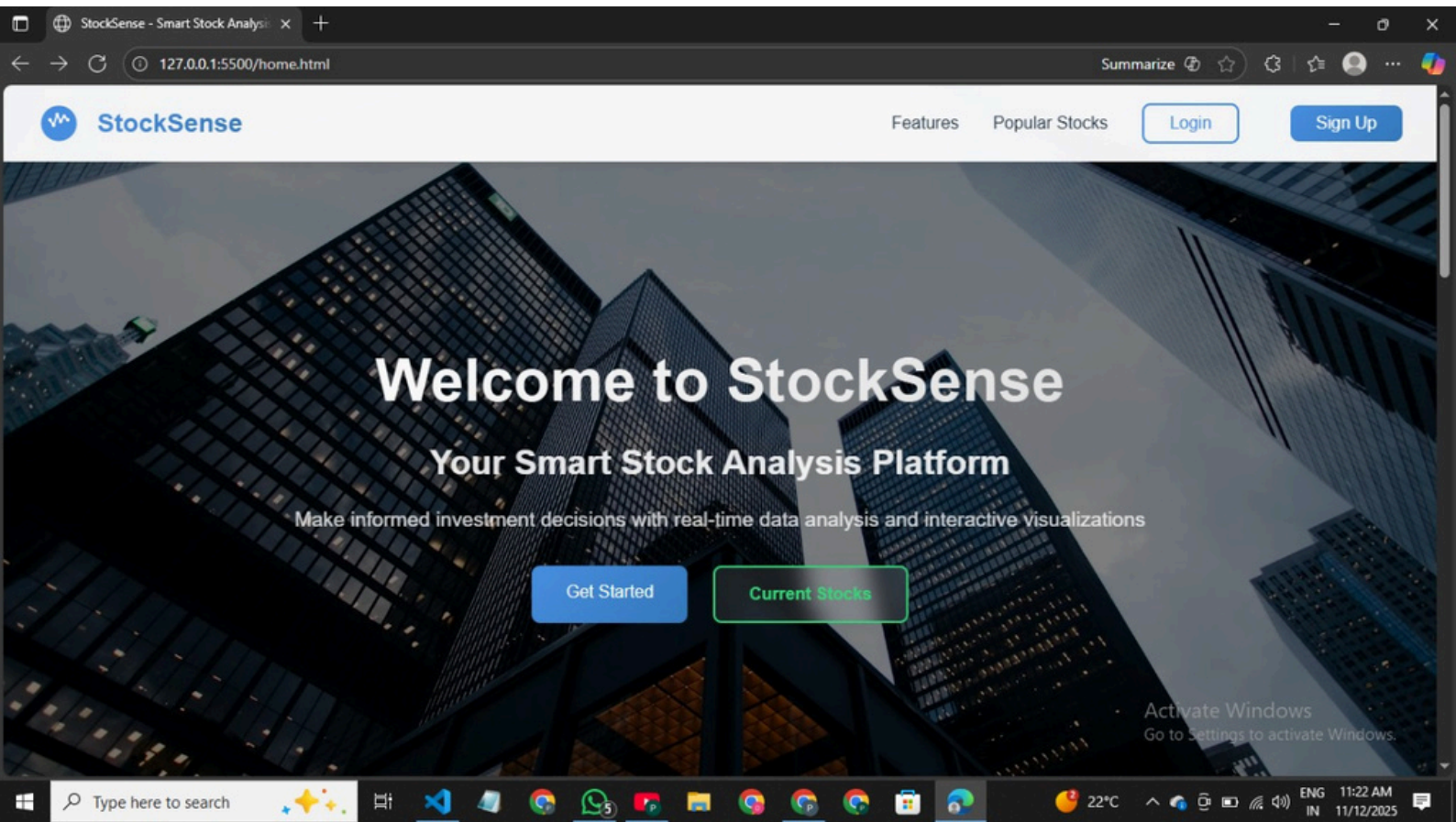


Figure 1.2: Sign-Up page of the StockSense application where new users can create an account by entering their name, email, and password. This interface connects to the backend database (users.db) to securely store user credentials for authentication.

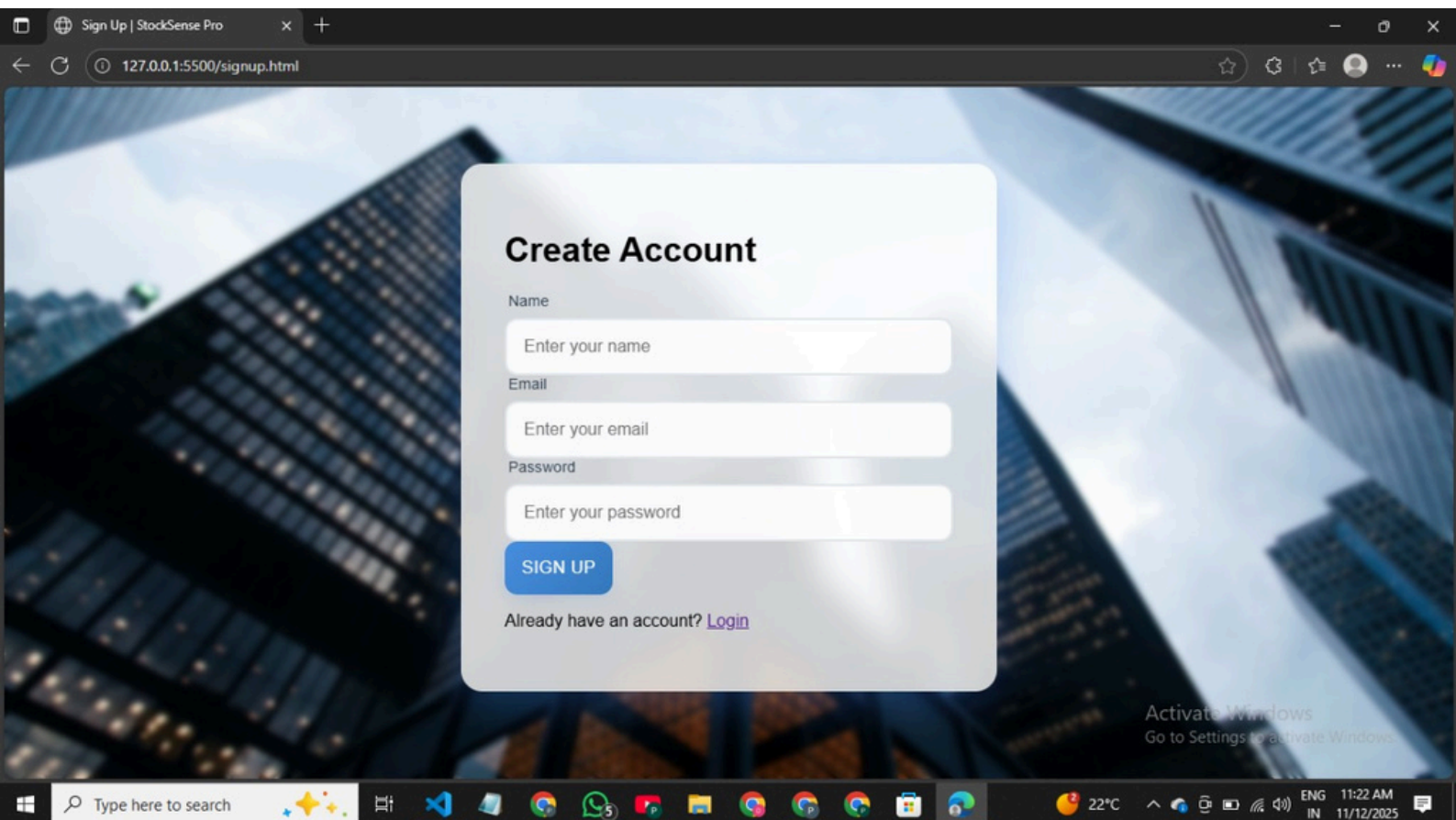


Figure 1.3: SQLite database (users.db) viewed in Visual Studio Code showing the users table where login and signup credentials (name, email, and password) are securely stored and managed for authentication purposes.

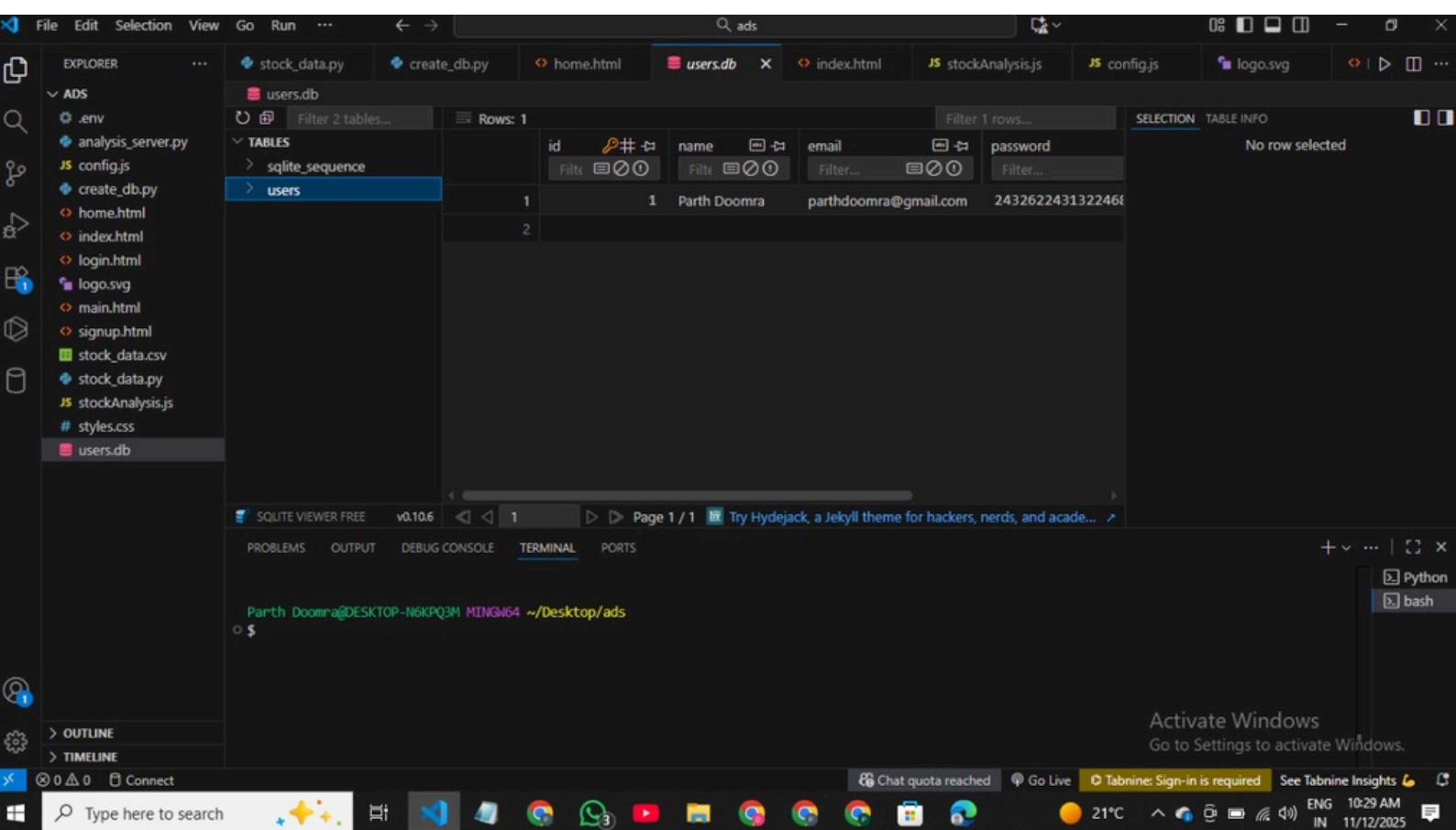


Figure 1.4: AI-powered StockSense Analytics Dashboard illustrating comparative stock trend visualization with dynamic data filtering and CSV integration.

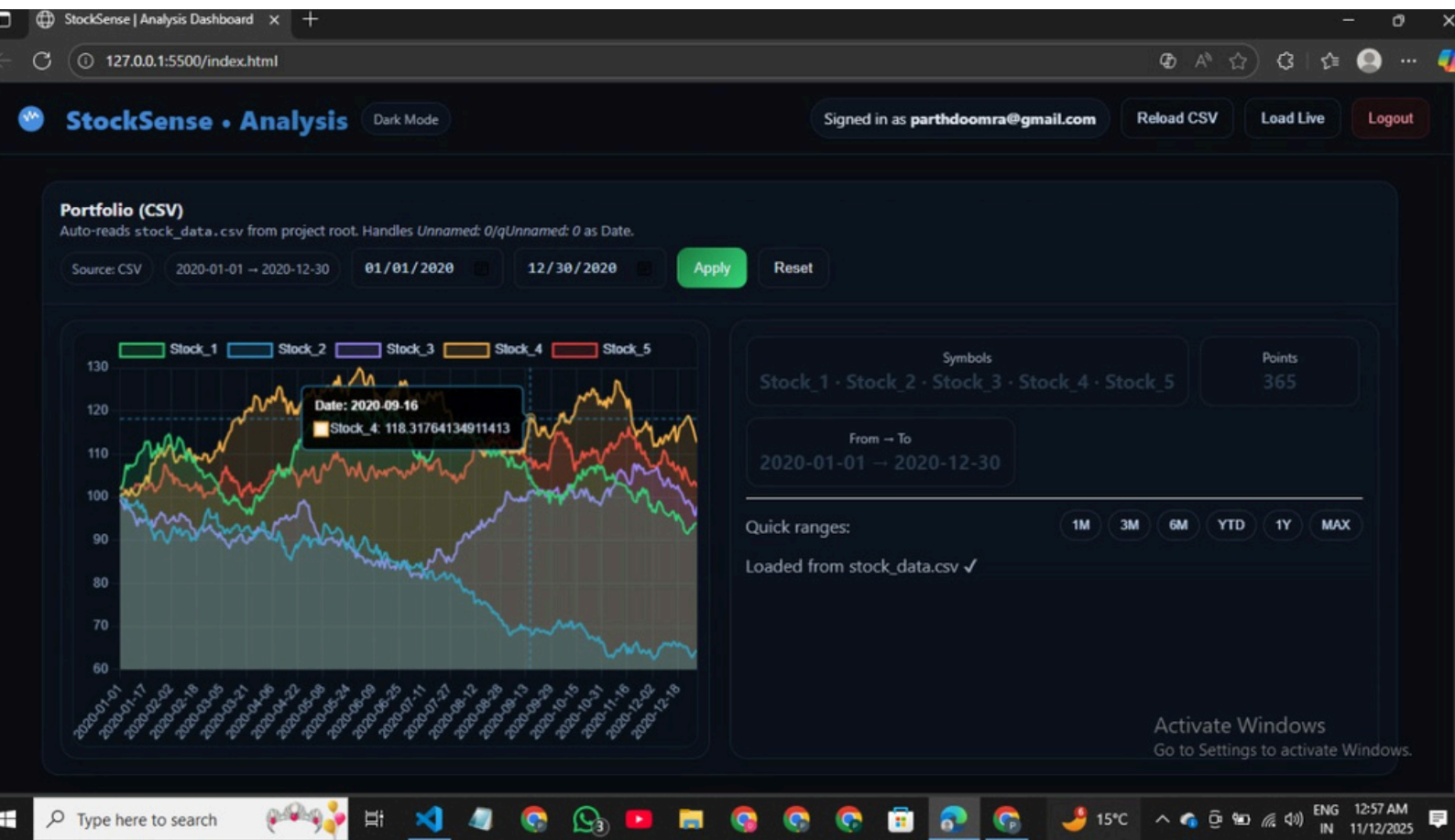
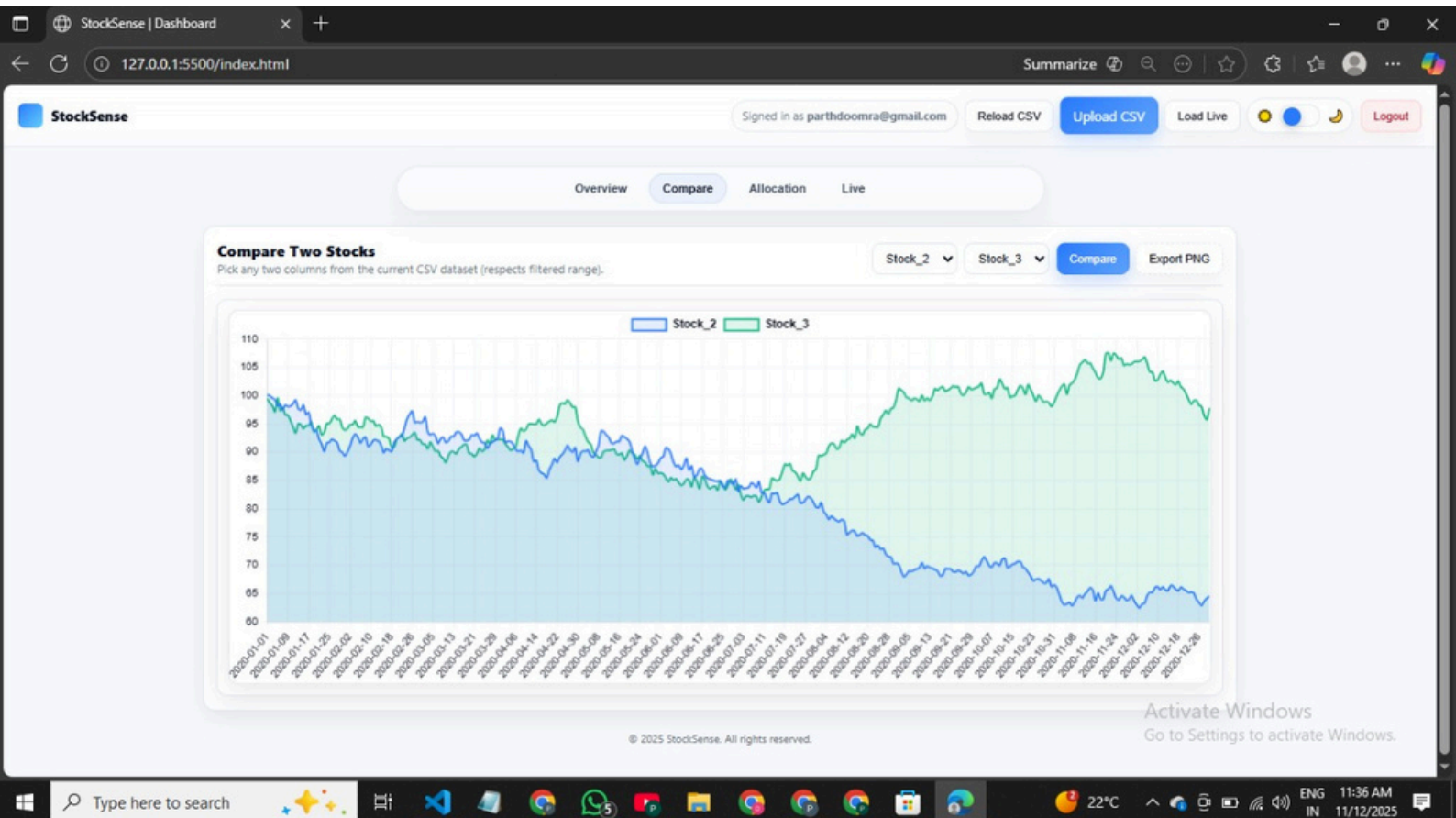
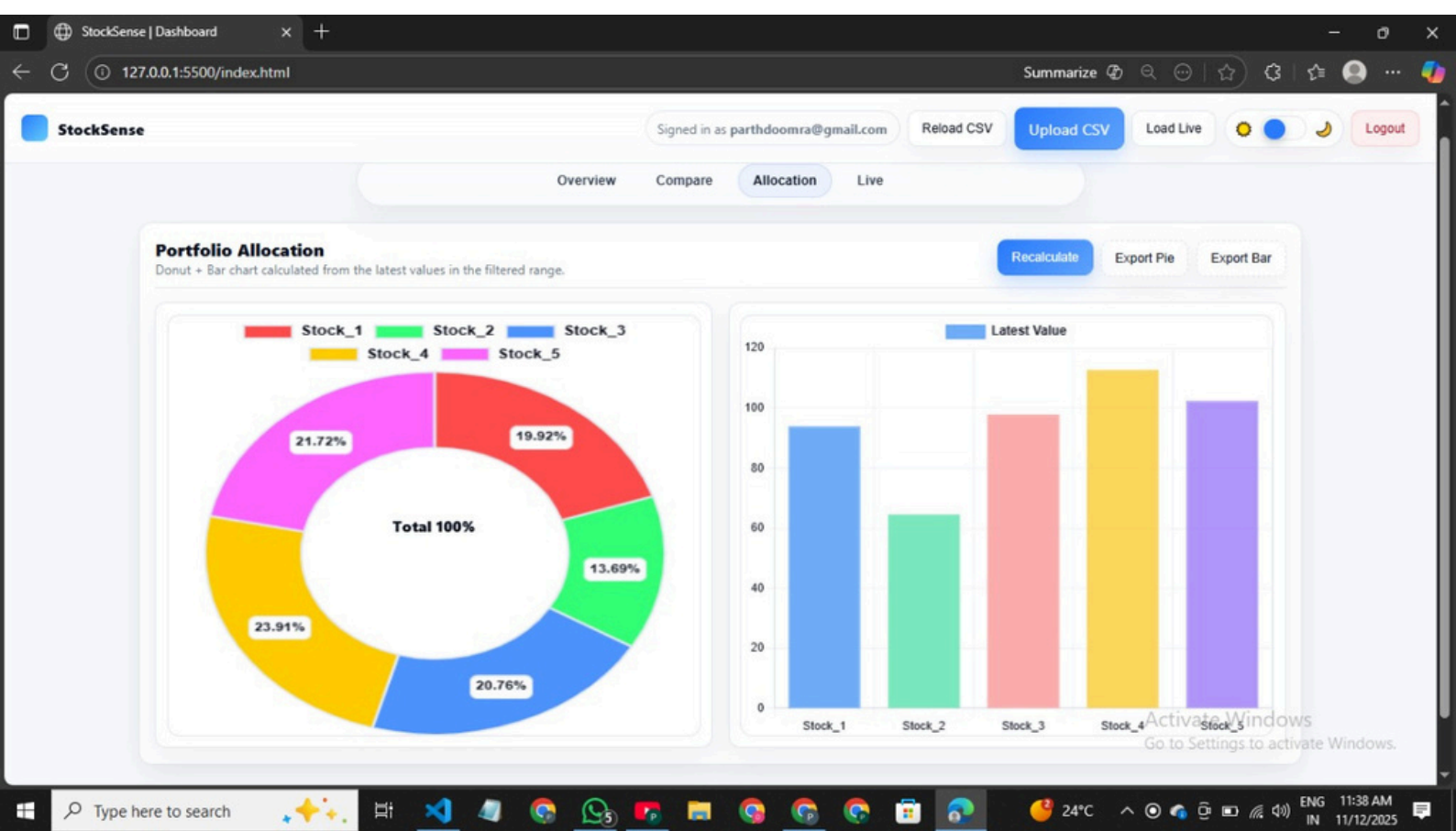


Figure 1.5: Dashboard view of the StockSense application comparing two selected stocks (Stock_2 and Stock_3) using an interactive graph. The interface allows users to upload datasets, filter date ranges, and export visual insights for portfolio analysis.



1.6: Portfolio Allocation view of the StockSense Dashboard showcasing a donut and bar chart that visualize the proportional distribution of multiple stocks based on their latest values. Users can recalculate, export pie charts, and export bar graphs for further analysis and reporting.



REFERENCES

•<https://flask.palletsprojects.com/> • <https://www.crummy.com/software/BeautifulSoup/> •
<https://pandas.pydata.org/> • <https://www.mysql.com/>

CONCLUSION

The system meets the primary objectives of automated data retrieval and analysis. The test cases provided cover typical user and admin workflows that should be validated before submission.

FUTURE ENHANCEMENTS

- Add automated unit tests and CI/CD integration.
- Implement end-to-end (E2E) UI tests with Selenium or Playwright.
- Add detailed performance/load testing for scraper and DB.
- Expand AI module to include model training and backtesting.